1 Einleitung

Das Zettabyte File System (ZFS) wurde ursprünglich von Sun Microsystems entwickelt und 2005 als Open-Source-Software veröffentlicht.[1] Im Unterschied zu klassischen Dateisystemen vereint die Software die Aufgaben eines Logical Volume Managers und eines Dateisystems in einer einzigen Schicht. Dadurch entfällt die traditionelle Trennung zwischen Partitionsverwaltung, RAID-Controller und Dateisystem.

Das zentrale Designprinzip von ZFS ist **Copy-on-Write** (CoW).[2] Anstatt Daten an ihrem bestehenden Speicherort zu verändern, schreibt ZFS modifizierte Blöcke stets an neue Positionen und aktualisiert die Metadaten erst, wenn der Schreibvorgang vollständig abgeschlossen ist. Damit ist jeder Schreibvorgang von Natur aus atomar: Ein Systemabsturz mitten in einer Operation hinterlässt keinen inkonsistenten Zustand.

Eng mit CoW verbunden ist das Konzept der **Snapshots**. Da ZFS Blöcke nie überschreibt, kann der Zustand eines Datasets zu einem beliebigen Zeitpunkt eingefroren werden, indem ein Verweis auf den aktuellen Wurzelblock gespeichert wird. Dieser Snapshot belegt zunächst keinen zusätzlichen Speicher. Erst wenn Daten nach der Erstellung verändert werden, wächst sein Platzbedarf um die Differenz zwischen altem und neuem Zustand.

Die Speicherverwaltung in ZFS basiert auf **Pools** und **Datasets**. Ein Pool fasst eine oder mehrere physische Disks zu einem gemeinsamen Speicherbereich zusammen und kann dabei verschiedene Topologien abbilden: einfaches Striping ohne Redundanz, gespiegelte Konfigurationen (Mirror) sowie RAIDZ. Innerhalb eines Pools werden Datasets angelegt, die sich Speicher dynamisch aus dem Pool teilen und jeweils eigene Eigenschaften wie Kompression, Deduplizierung oder Quotas erhalten können.

Zur Verbesserung der Leseperformanz verwendet ZFS den **Adaptive Replacement Cache** (ARC). Dabei handelt es sich um einen Hauptspeicher-Cache, der unabhängig vom Linux Page Cache betrieben wird. Der ARC unterscheidet zwischen häufig genutzten (*frequently used*) und zuletzt genutzten (*recently used*) Blöcken und passt seine Ersetzungsstrategie dynamisch an das Zugriffsverhalten an.

Das vorliegende Portfolioabgabe untersucht ZFS anhand von vier praktischen Aufgaben.

2 Umgebung und Ansatz

Die Aufgaben wurden auf einer Ubuntu-24.04-LTS VM (KVM-Hypervisor) mit ZFS 2.2.2 implementiert. Als Speichermedien dienen vier 500 MB große Sparse-Image-Dateien.

Die Implementierungssprache ist Python 3.12. Um ZFS-Kommandos konsistent zu kapseln, wurde eine objektorientierte Klassenhierarchie mit `ShellCommander` als gemeinsamer Basisklasse entwickelt. Jede abgeleitete Klasse kapselt eine logische ZFS-Einheit und exponiert die wichtigsten ZFS-Methoden. Die Hierarchie umfasst die Klassen `ZFSDisk`, `ZFSPool`, `ZFSDataset`, `ZFSSnapshot`, `BackupManager`, `Ext4Disk` sowie `Ext4Mount`.

[1] Carol Sliwa. *ZFS*. `https://www.computerweekly.com/de/definition/ZFS`. 2022.
[2] Wikipedia contributors. *ZFS*. `https://en.wikipedia.org/wiki/ZFS`. 2025.

3 Aufgaben

3.1 Datei-Inkonsistenzen trotz ZFS-Snapshots

Für das Experiment werden zwei ZFS-Pools angelegt: `inconsistpool` auf `disk1` als Quelle und `recvpool` auf `disk2` als Empfänger. Innerhalb von `inconsistpool` wird über `pool.get_dataset()` ein Dataset erzeugt, der Ablageort für `counter.txt` dient.

Ein separater Writer-Thread schreibt in einer Endlosschleife nicht-atomar in diese Datei. Jeder Datensatz entsteht in zwei getrennten Schritten: Zunächst wird nur der Zähler `count=N` in die Datei geschrieben und sofort per `flush()` auf die Disk übertragen. Anschließend wartet der Thread 10 ms, bevor er eine Prüfsumme `checksum=sha256(N)` als Fortsetzung derselben Zeile ergänzt. In diesem 10 ms-Fenster ist der Record auf der Disk vorhanden, aber semantisch unvollständig. Genau hier kann ein Snapshot eintreten.

Nach 2 Sekunden wird über `ds.snapshot()` der Snapshot `snap-inconsist` erstellt und per `snap.pipe()` direkt in das Dataset `recvpool/inspect` übertragen. Intern startet `pipe()` zwei verkettete Subprozesse: `zfs send` schreibt den Snapshot-Stream, den `zfs receive` direkt liest und in den Empfänger-Pool schreibt. Anschließend wird die letzte Zeile der `counter.txt` im empfangenen Dataset mit dem aktuellen Stand der Originaldatei verglichen, um festzustellen, ob der Snapshot das Inkonsistenzfenster getroffen hat.

Das Experiment ist nicht deterministisch, da das Timing des Snapshots variiert. Im inkonsistenten Fall ergibt sich eine Ausgabe ähnlich zu dieser:

```
============================================================
INCONSISTENT: last line was cut mid-write
  Inconsistent last line: 'count=192'
  Clean last line:        'count=954  checksum=b47ae...3f\n'
============================================================
```

ZFS garantiert **Crash-Konsistenz**: Entweder ist ein Block vollständig auf Disk, oder gar nicht. ZFS garantiert jedoch **keine Applikationskonsistenz**. Der Snapshot fixiert den Dateisystemzustand zu einem exakten Zeitpunkt, unabhängig davon, ob die Applikation einen logischen Datensatz vollständig geschrieben hat.

Die Ausgabe des Experiments belegt dies konkret. Der Snapshot traf in das 10 ms-Fenster zwischen den beiden Phasen und fror dabei nur `count=192` ein und die Prüfsumme war zu diesem Zeitpunkt noch nicht auf die Disk geschrieben worden. Im Original lief der Writer derweil weiter und hatte zum Zeitpunkt der Auswertung bereits Record 954 vollständig abgeschlossen. Der Unterschied von 762 Records entspricht den ca. 8 Sekunden, die zwischen Snapshot-Zeitpunkt und dem Ende des Writer-Threads lagen.

Um applikationskonsistente Snapshots zu gewährleisten, existieren mehrere Ansätze. Beim atomaren Schreiben wird die Datei zunächst in eine temporäre Datei geschrieben und anschließend via `rename()` ausgetauscht, sodass stets die alte oder die neue Version sichtbar ist. Alternativ kann die Applikation per `SIGSTOP` pausiert, der Snapshot aufgenommen und anschließend per `SIGCONT` fortgesetzt werden. Datenbanken wie PostgreSQL oder MySQL bieten darüber hinaus eigene Mechanismen wie `CHECKPOINT` oder `FLUSH TABLES WITH READ LOCK`, um vor dem Snapshot einen konsistenten Zustand zu erzwingen.

3.2 Datensicherheit mit RAIDZ

Ein RAIDZ-Pool `raidpool` wird über drei Disks (`disk1`, `disk2`, `disk3`) angelegt, sodass ZFS Daten und Paritätsinformation gemeinsam über alle drei Disks striped. `disk4` steht als Ersatz-Disk bereit. Innerhalb des Pools wird das Dataset `raidpool/data` erzeugt, in dem die Datei `workload.txt` abgelegt wird. Ein Worker-Thread liest und beschreibt diese Datei in einer Endlosschleife mit ca. 20 Iterationen pro Sekunde. Tritt dabei ein I/O-Fehler auf, wird ein Event gesetzt und der Thread bricht ab. Andernfalls läuft er bis zum expliziten Stop weiter.

Nach 3 Sekunden normalem Betrieb wird `disk1` über `disk.fail()` ein Ausfall simuliert, indem die zugrundeliegende Image-Datei auf 0 Byte truncated wird. ZFS erkennt den Datenverlust auf Blockebene und versetzt den Pool in den DEGRADED-Zustand. Der Worker läuft in dieser Phase unverändert weiter. Nach weiteren 5 Sekunden wird über `disk.replace()` der Befehl `zpool replace` ausgelöst, der `disk4` als Ersatz einbindet und das Resilvering startet: ZFS rekonstruiert die fehlenden Blöcke aus den verbleibenden Disks mithilfe der Paritätsinformation und schreibt sie auf disk4. Nach weiteren 3 Sekunden wird der Worker gestoppt und der finale Poolzustand ausgewertet. Der gesamte Versuch dauert damit ca. 11 Sekunden, während derer der Pool durchgehend les- und schreibbar bleibt und keine Fehler geworfen werden.

Der Worker hat 219 Iterationen ohne einen einzigen I/O-Fehler abgeschlossen. Während der gesamten DEGRADED-Phase blieb das Dataset uneingeschränkt les- und schreibbar. Die finale Zusammenfassung des Programms bestätigt dies:

```
============================================================
  SUMMARY
============================================================
  Total iterations completed : 219
  I/O errors during failure  : NO  -- RAIDZ kept data
     accessible
  Final pool status:
  pool: raidpool
 state: ONLINE
  scan: resilvered 180K in 00:00:00 with 0 errors
        on Sat Mar 14 17:20:03 2026
config:
    NAME                                STATE  READ WRITE CKSUM
    raidpool                            ONLINE    0     0     0
      raidz1-0                          ONLINE    0     0     0
        /2025WEdition/disks/disk4.img   ONLINE    0     0     0
        /2025WEdition/disks/disk2.img   ONLINE    0     0     0
        /2025WEdition/disks/disk3.img   ONLINE    0     0     0
errors: No known data errors
============================================================
```

3.3 Zugriffsperformanz: ZFS vs. ext4

Für den Vergleich werden zwei unabhängige Setups erstellt. Für ZFS wird über `ZFSPool` ein Pool `benchpool` auf `disk1` angelegt und darin über `pool.get_dataset()` ein Dataset erzeugt. Für ext4 wird `disk2` über `Ext4Disk.format()` mit `mkfs.ext4` formatiert. Anschließend bindet `Ext4Disk.attach()` die Image-Datei als Loop-Device ein und gibt ein `Ext4Mount`-Objekt zurück, über das `mount()` das Dateisystem einhängt.

Alle fünf Benchmark-Tests erben von einer gemeinsamen Basisklasse und implementieren jeweils eine `run()`-Methode, die einen Mountpunkt und ein Dateisystemkürzel entgegennimmt und ein Ergebnis mit Throughput und Einheit zurückgibt. Sequential Write und Sequential Read arbeiten mit 256 MB großen Dateien und 1 MB-Blöcken. Die Random-Tests führen jeweils 500 Zugriffe auf 4 KB-Blöcke durch, wobei Random Write nach jedem Zugriff ein `fsync()` ausführt. Der Small-Files-Test legt 500 Dateien à 4 KB an und synchronisiert jede einzeln. Um den ZFS ARC vor den Lesetests zu invalidieren, wird `zfs_arc_max` vor jedem Lesetest temporär auf ein Minimum gesetzt und danach wieder zurückgesetzt. Die Tests laufen nacheinander auf beiden Mountpunkten ab.

```
============================================================
  Benchmark Results: ZFS vs ext4
============================================================
Test                                ZFS          ext4
------------------------------------------------------------
Sequential Write            228.0 MB/s    652.3 MB/s
Sequential Read            5433.4 MB/s   2782.0 MB/s
Random Write (4K)             1.9 MB/s      0.6 MB/s
Random Read  (4K)            82.2 MB/s     20.7 MB/s
Small Files              531.5 files/s 157.9 files/s
============================================================
```

Die gemessenen Werte sind auf Basis der eingesetzten Hardware plausibel und können als valide eingestuft werden. Als Speichermedium kommt eine SK Hynix PCIe-4.0-NVMe zum Einsatz, die laut Herstellerangaben sequentielle Leseraten von ca. 5.000–6.500 MB/s und sequentielle Schreibraten von ca. 4.000–5.000 MB/s erreicht.

Die deutlich niedrigeren sequentiellen Schreibraten von 228 MB/s (ZFS) bzw. 652 MB/s (ext4) im Vergleich zu den theoretischen NVMe-Werten erklären sich durch mehrere zusammenwirkende Faktoren. Erstens erzwingt ein abschließendes `os.fsync()`, dass alle Daten physisch auf das Speichermedium geschrieben sein müssen, bevor die Messung endet. Zweitens verläuft der I/O-Pfad in der VM-Umgebung mehrfach verschachtelt: Schreibzugriffe durchlaufen ZFS bzw. ext4 innerhalb der VM, das darunterliegende Loop-Device, die Sparse-Image-Datei und schließlich den KVM-Hypervisor mit seinem eigenen Host-Dateisystem, bevor die NVMe erreicht wird.

Für den Vergleich der relativen Stärken beider Dateisysteme sind die Messwerte dennoch aussagekräftig: ZFS zeigt bei Random-Zugriffen und kleinen Dateien durch den ARC einen deutlichen Vorteil gegenüber ext4, während ext4 beim sequentiellen Schreiben durch das einfachere In-Place-Write-Modell und geringeren Verwaltungsaufwand überlegen ist.